

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions.

1. Identity Kernel

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

2. Sobel-X Kernel (Vertical Edge Detection)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

3. Sobel-Y Kernel (Horizontal Edge Detection)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

4. Laplacian Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

5. Sharpening Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

6. Box Blur Kernel

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

7. Gaussian Blur Kernel

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

8. Emboss Kernel

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

9. Outline Detection Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

10. Motion Blur Kernel

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output positions: $1(1)+2(0)+4(0)+5(1) = 6$, $2(1)+3(0)+5(0)+6(1) = 8$, $4(1)+5(0)+7(0)+8(1) = 12$, $5(1) + 6(0) + 8(0) + 9(1) = 14$

$$\text{Feature Map} = \begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

$$\text{Input} = \begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix} \quad \text{Output} = \begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Input: $32 \times 32 \times 3$, Conv layer: 16 filters of 3×3 .

$$(3 \times 3 \times 3 + 1) \times 16 = 28 \times 16 = 448 \text{ parameters}$$

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1 – Dataset Exploration

- Load CIFAR-10 and display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2 – Convolution Layer

Kernel sizes compared: 3×3 , 5×5 , 7×7 (VALID padding, stride 1).

Kernel Size	Output Feature Map Shape
3×3	(1, 30, 30, 1)
5×5	(1, 28, 28, 1)
7×7	(1, 26, 26, 1)

Task 3 – Stride and Padding Study

Stride	Padding	Output Shape
1	VALID	(1, 30, 30, 1)
1	SAME	(1, 32, 32, 1)
2	VALID	(1, 15, 15, 1)
2	SAME	(1, 16, 16, 1)

Task 4 – Feature Map Visualization

8 feature maps visualized after the first Conv2D layer (filters=8, kernel 3×3 , ReLU).

Task 5 – Max Pooling vs Average Pooling

Pooling Type	Test Accuracy	Test Loss
Max Pooling	0.6167	1.1135
Average Pooling	0.5869	1.1842

Task 6 – CNN Architecture

Input $\rightarrow$ Conv32 $\rightarrow$ MaxPool $\rightarrow$ Conv64 $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense64 $\rightarrow$ Softmax

Trained for 10 epochs, batch size 64, optimizer Adam.

Task 7 – Model Evaluation: See Section 11 (Results).

9. Mandatory Plots

9.1 Sample Images

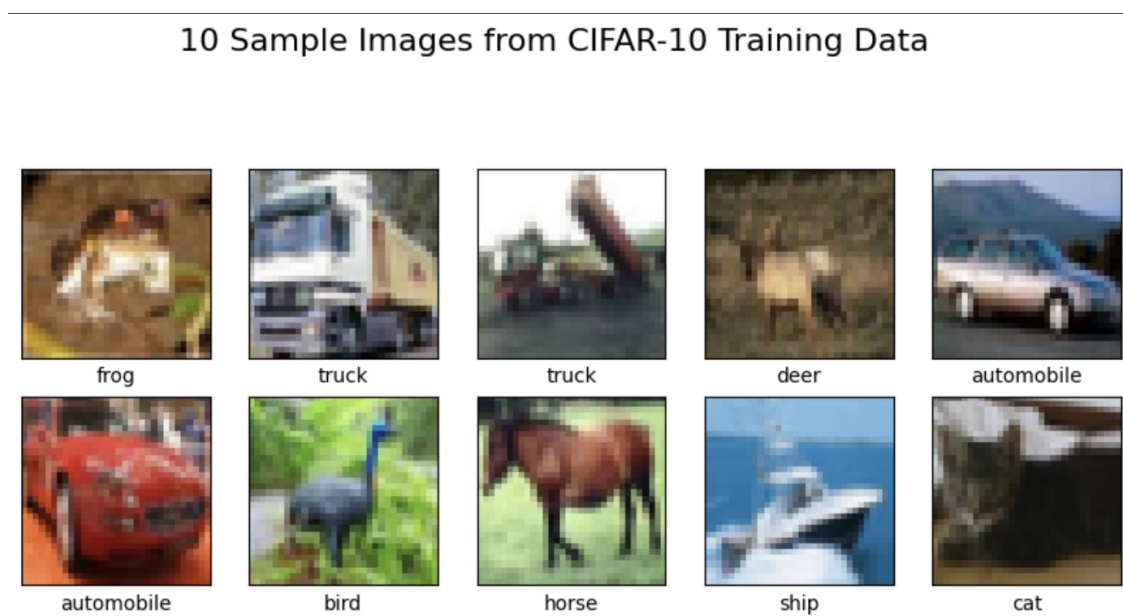


Figure 1: Ten sample images from the CIFAR-10 training set with class labels.

Inference: The dataset has 10 visual classes/categories at 32x32 resolution.

9.2 Class Distribution

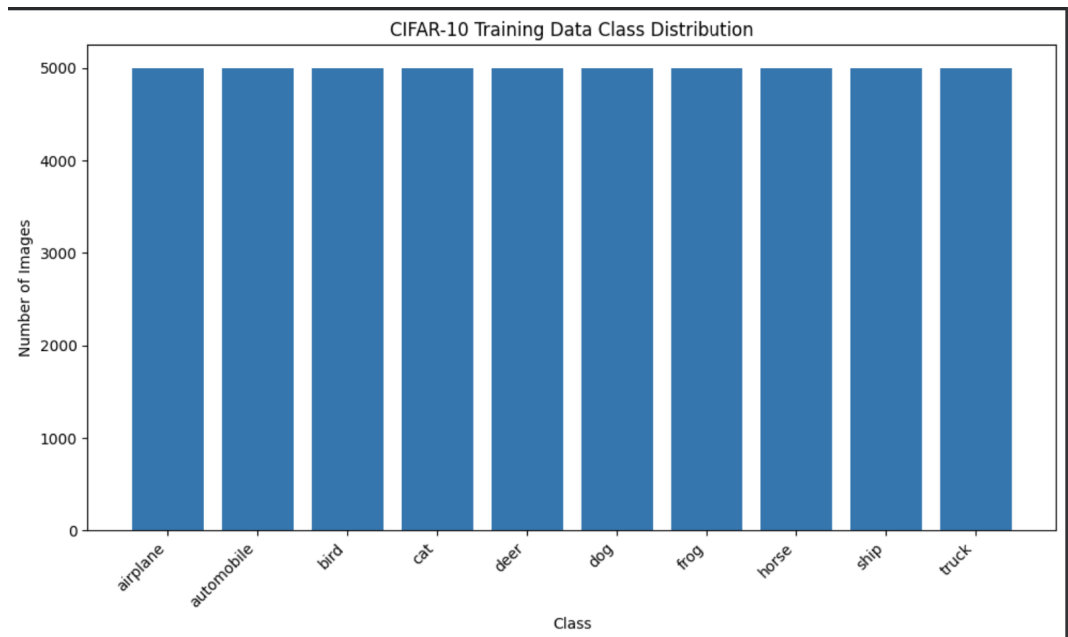


Figure 2: Class distribution of the CIFAR-10 training set.

Inference: The CIFAR-10 dataset is perfectly balanced with 5000 images per class with no chance of class imbalance existing within the training set.

9.3 Training Accuracy

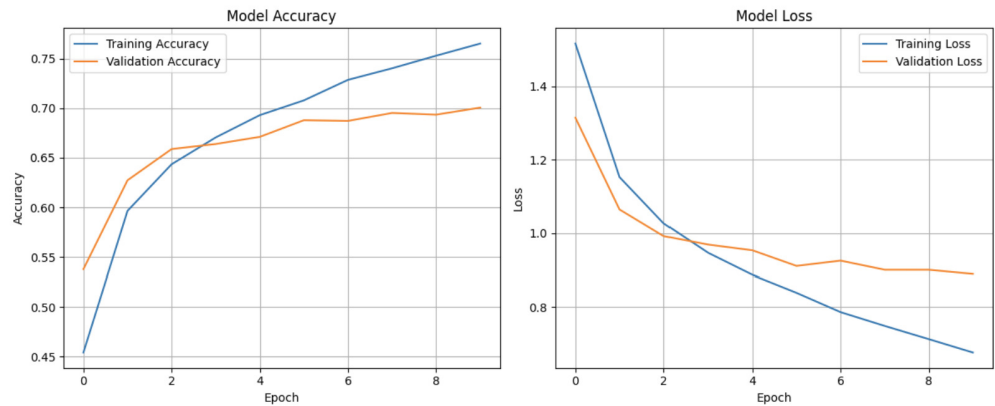


Figure 3: Training accuracy over 10 epochs.

Inference: There are 2 graphs: model accuracy and loss each having training and validation metrics. Training increases from 0.45 from epoch 1 to 0.76 at epoch 10 with no sudden jumps - loss also steadily decreases from 1.4 to less than 0.3 after epoch 8.

9.4 Validation Accuracy

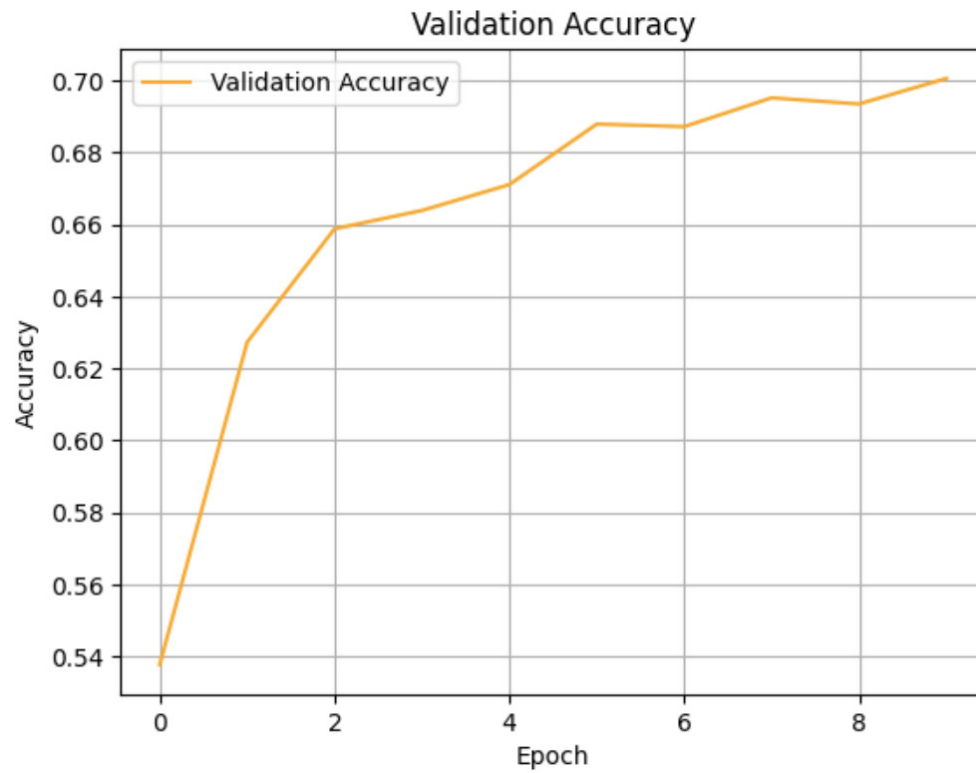


Figure 4: Validation accuracy over 10 epochs.

Inference: Validation accuracy has a sharp increase from epochs 0-2 and plateaus at 0.70 from epoch 5 onwards with the formation of a gap between training and validation indicating mild overfitting and memorization done by the model so the training is kept at 10 epochs.

9.5 Training Loss

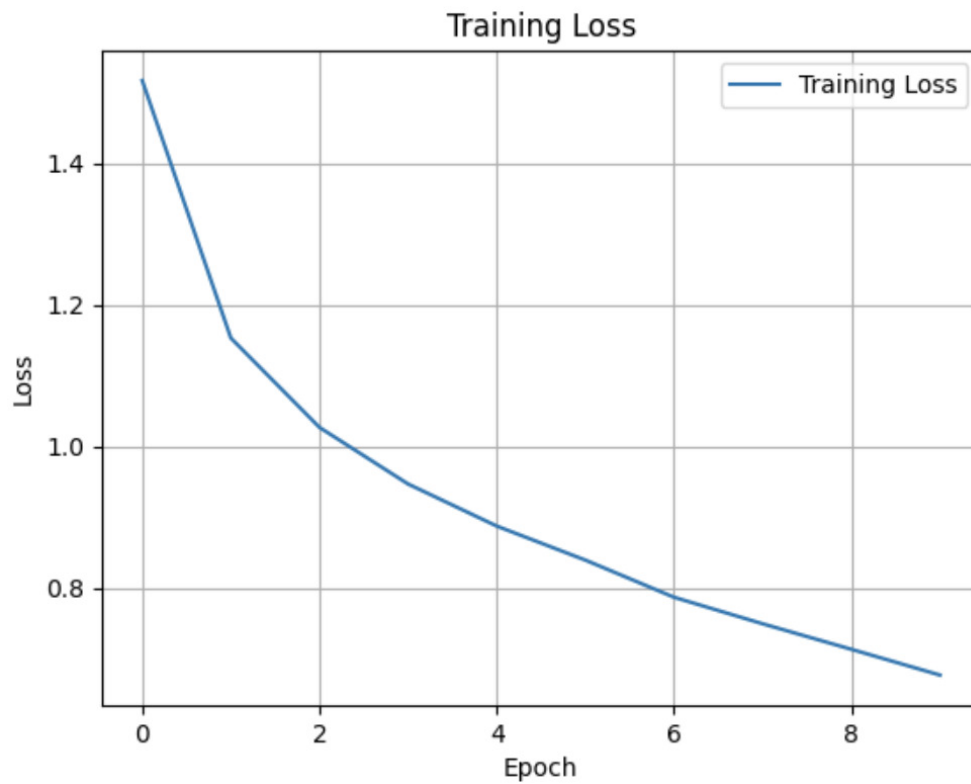


Figure 5: Training loss over 10 epochs.

Inference: Training loss decreases from 1.52 to 0.68 indicating that the optimiser is effective at reducing the objective function loss - with a steep dip from epoch 0-2 that shows rapid initial feature learning.

9.6 Validation Loss

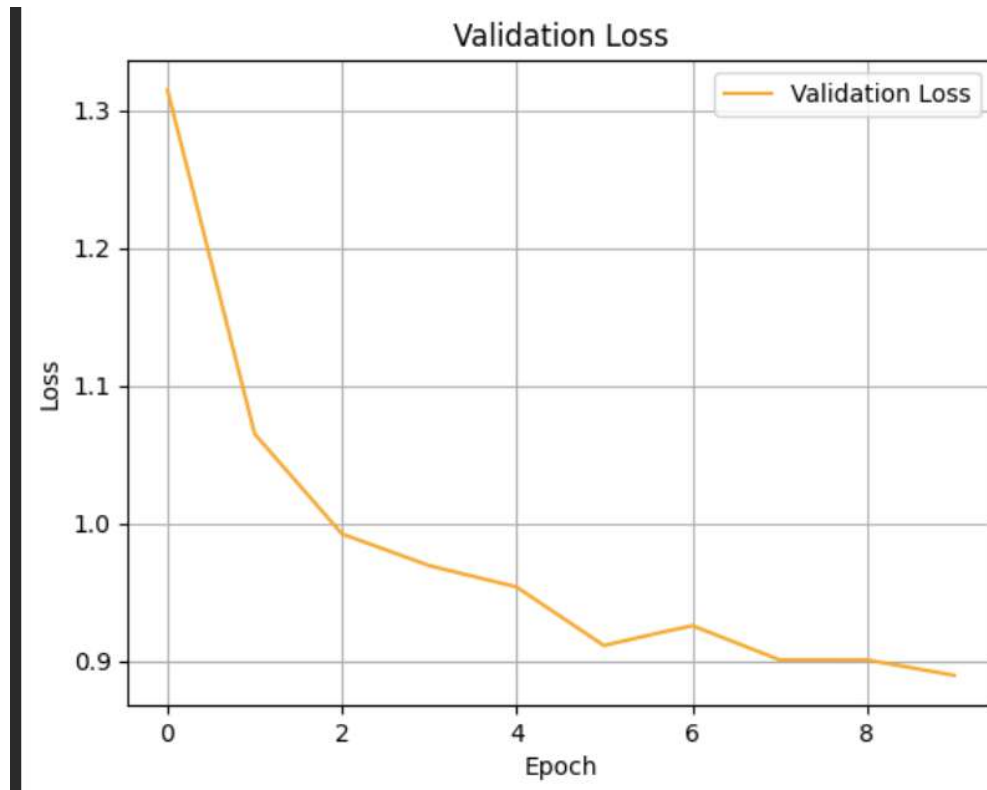


Figure 6: Validation loss over 10 epochs.

Inference: Validation loss decreases sharply until epoch 5 and then plateaus around 0.89 to 0.9 with a divergence between this and training loss indicating mild overfitting so dropout could help here.

9.7 Feature Maps

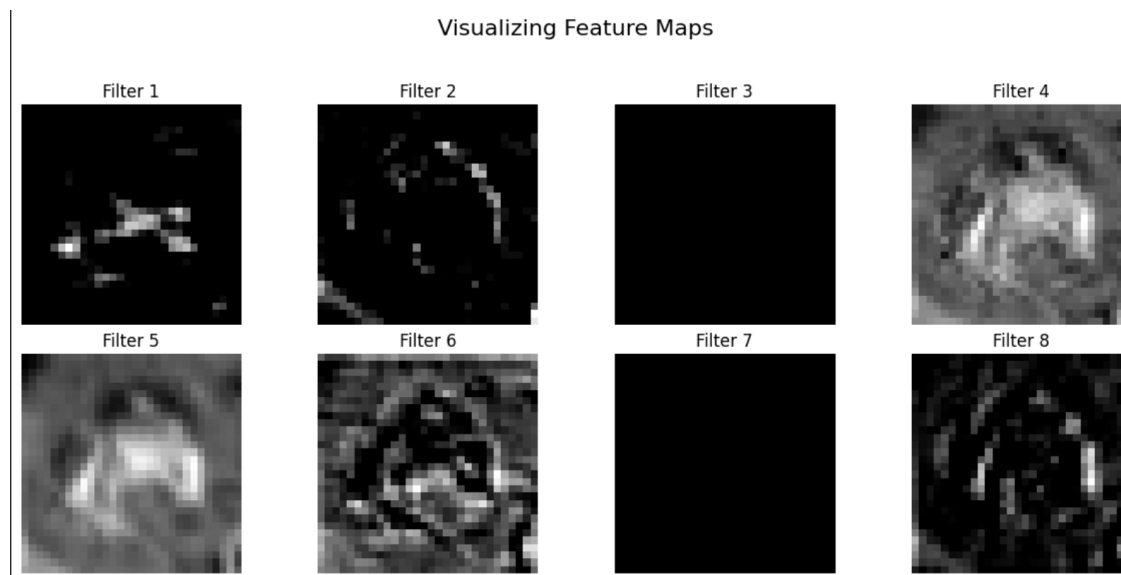


Figure 7: Eight feature maps produced by the first Conv2D layer on a sample CIFAR-10 image.

Inference: Filters 1, 2, 4, 5, 6, and 8 activate on edges, textures, and shapes, while filters 3 and 7 produce entirely black outputs. The black feature maps indicate those filters did not find any matching patterns in the sample image with their randomly initialized weights. This demonstrates that not all filters learn equally useful representations, especially before training.

9.8 Confusion Matrix

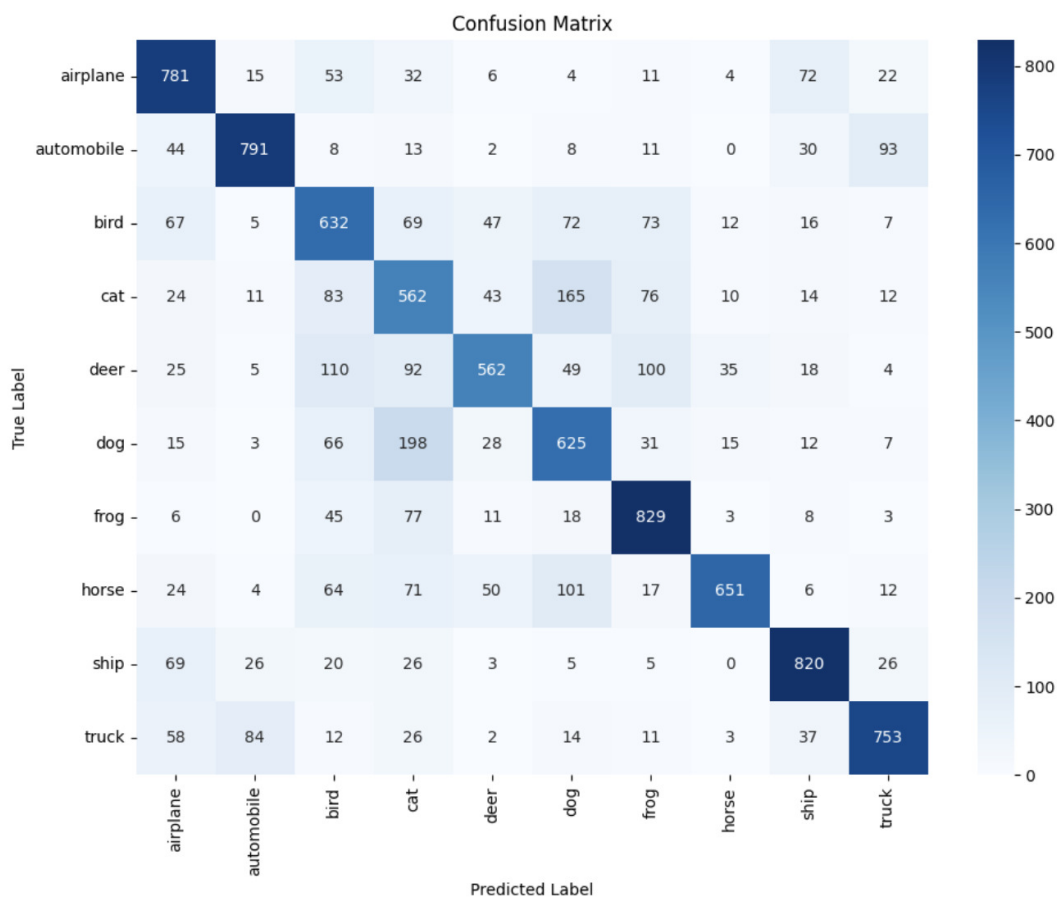


Figure 8: Confusion matrix for the CNN model on the CIFAR-10 test set.

Inference: The model's best performance happens with the frog class, followed by automobile and ship. The model seems to get confused between cat and dog maybe because of their visual similarity.

9.9 Pooling Comparison

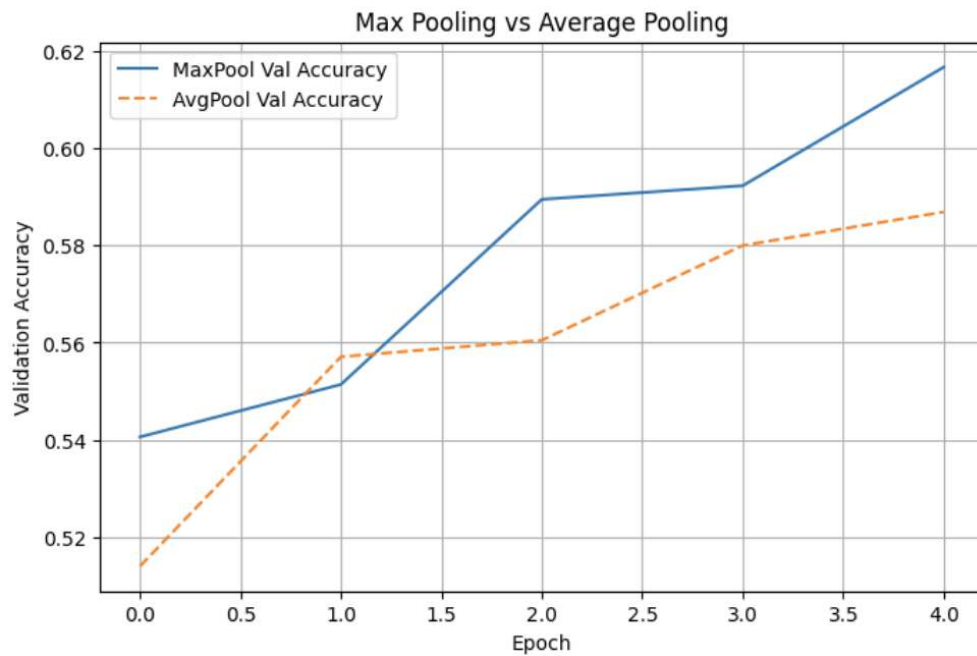


Figure 9: Validation accuracy comparison: Max Pooling vs Average Pooling over 5 epochs.

Inference: Max pooling seems to consistently outperform average pooling across 5 test epochs - with max pooling preserving dominant features like edges so this maybe useful for object recognition while average pooling smooths things out therefore there is considerable information loss.

9.10 Sample Predictions

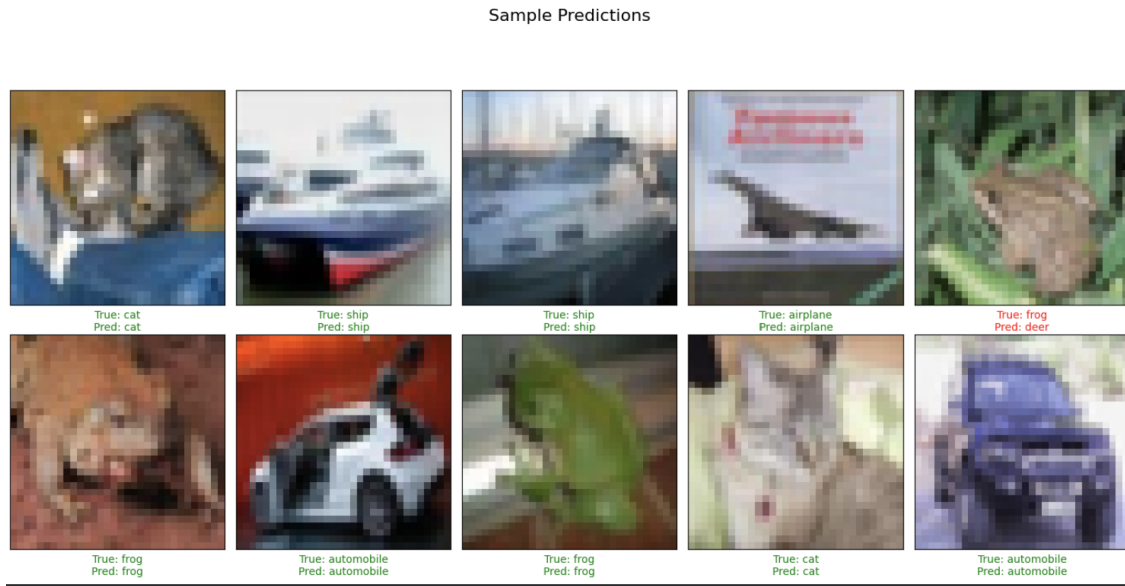


Figure 10: Sample test predictions. Green labels indicate correct predictions; red labels indicate incorrect predictions.

Inference: The model classifies 9/10 sample test images with one misclassification - frog as deer. The single error maybe because the model seems to be confused with green toned backgrounds so further visual data augmentations can be used.

10. Training History

Epoch	Train Acc	Train Loss	Val Acc	Val Loss
1	0.4542	1.5169	0.5377	1.3148
2	0.5965	1.1531	0.6273	1.0649
3	0.6435	1.0268	0.6588	0.9923
4	0.6705	0.9468	0.6639	0.9693
5	0.6930	0.8874	0.6711	0.9538
6	0.7079	0.8393	0.6879	0.9112
7	0.7286	0.7868	0.6872	0.9257
8	0.7403	0.7493	0.6952	0.9008
9	0.7530	0.7133	0.6935	0.9008
10	0.7652	0.6769	0.7006	0.8896

11. Results

11.1 Performance Metrics

Metric	Value
Training Accuracy (epoch 10)	76.52%
Testing Accuracy	70.06%
Precision (macro avg)	0.71
Recall (macro avg)	0.70
F1-score (macro avg)	0.70
Training Time (per epoch)	$\approx 60\text{--}84$ s

11.2 Classification Report

Class	Precision	Recall	F1-score	Support
Airplane	0.70	0.78	0.74	1000
Automobile	0.84	0.79	0.81	1000
Bird	0.58	0.63	0.60	1000
Cat	0.48	0.56	0.52	1000
Deer	0.75	0.56	0.64	1000
Dog	0.59	0.62	0.61	1000
Frog	0.71	0.83	0.77	1000
Horse	0.89	0.65	0.75	1000
Ship	0.79	0.82	0.81	1000
Truck	0.80	0.75	0.78	1000
Accuracy	0.70			10000
Macro avg	0.71	0.70	0.70	10000
Weighted avg	0.71	0.70	0.70	10000

12. Discussion

1. Why is convolution preferred over fully connected layers for images?

Fully connected layers have to connect every input pixel to every neuron - giving 3072 inputs and this maybe a cause for exploding gradients wwith larger images. Convolution solves this using a small shared kernel which slides across the image - reducing the number of weights used.

2. How does stride affect the feature map size?

Stride controls how many pixels the kernel moves at each step. With stride 1 on a 32×32 image using a 3×3 kernel (VALID), output is 30×30 . With stride 2, the kernel skips every other position giving 15×15 — roughly half the size. Larger stride means smaller feature maps, faster computation, but loss of fine spatial detail.

3. What is the role of padding?

VALID makes sure the feature map shrinks after each convolution whereas SAME adds zeros around the border so spatial size remains the same - padding ensures that dimensions are preserved and the pixels in the edge have the same amount of coverage as center pixels.

4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps, cutting computation and memory requirements in deeper layers — if a detected feature shifts by a pixel, max pooling still captures it. It also acts as a regularizer of sorts, by discarding less important spatial information, helping the model generalize better

5. How do feature maps represent image characteristics?

Each filter in the convolution layer learns to respond to a specific low-level pattern — edges in a particular direction, color gradients, or textures. The feature map shows where in the image that pattern is present (bright) or absent (dark). As you go deeper in the network, filters combine these low-level patterns into higher-level representations like shapes and object parts.

6. Why do CNNs require fewer parameters than MLPs?

Two mechanisms achieve this. First, weight sharing — the same kernel weights are reused at every spatial position rather than having unique weights per pixel connection. Second, local connectivity — each neuron only sees a small receptive field rather than the entire image. A 3×3 filter on a 3-channel image has only $27 + 1 = 28$ parameters regardless of image size, whereas a fully connected layer on the same image would need thousands.

13. Conclusion

This experiment implemented a CNN on CIFAR-10 and achieved a test accuracy of 0.7006 using two convolutional layers with max pooling and the Adam optimizer. Feature map visualization confirmed that different filters detect distinct spatial patterns such as edges and textures. Max pooling outperformed average pooling (0.6167 vs 0.5869) by preserving dominant activations. Visually similar classes like cat and dog showed the highest confusion, while structurally distinct classes like ship and automobile performed best. A mild overfitting trend was observed in later epochs, suggesting future improvements through dropout or data augmentation

14. References

1. Goodfellow et al., *Deep Learning*. MIT Press, 2016.
2. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
3. Haykin, *Neural Networks and Learning Machines*. Pearson, 2009.
4. TensorFlow Documentation. <https://www.tensorflow.org>
5. CIFAR-10 Dataset. <https://www.cs.toronto.edu/~kriz/cifar.html>